

Context API no React: Compartilhamento de Dados Entre Componentes.

Reaghrachng Component Integration

Reedte Colinein Tili YemjeadbtVerobus tedte totbimrogatereang



Component Library

Contraheis sis tooreed tar inger
corpesiv eormetomist ogia ture
icsetitetteuocorañh sceect.
oteneis, isineat qutibi totis.



Data Pipelines

Estuam topericcoemitt aminers
perumet maoobsemet oitette
aeoicaiolietosecoctelefoant,
connoeatual.



Integration Tools

Pec tamsat eproinne unt artliat ior
peneasatit melleonesatredata
ndineatit annoest connoatit.
uito dieens.

O que é a Context API?

A Context API é uma funcionalidade nativa do React que permite **compartilhar dados entre componentes** sem a necessidade de passar props manualmente através de cada nível da árvore de componentes.

Ela resolve o problema conhecido como "**prop drilling**" - quando você precisa passar props por vários níveis de componentes que não precisam desses dados, apenas para alcançar um componente filho distante.

A Context API é ideal para gerenciar **informações globais** que muitos componentes podem precisar acessar.



Quando Usar a Context API



Tema da Aplicação

Gerenciar temas claros/escuros e configurações visuais que afetam toda a interface.



Dados do Usuário

Compartilhar informações do usuário logado, preferências e permissões em toda a aplicação.



Internacionalização

Gerenciar o idioma selecionado e fornecer traduções para todos os componentes.



Carrinho de Compras

Manter o estado do carrinho acessível em múltiplas páginas e componentes.

Criando e Utilizando um Contexto

O que é um Contexto?

Um contexto é um **objeto criado com createContext()** que armazena um valor (estado ou qualquer outro dado) que pode ser **consumido por qualquer componente** dentro do escopo de um Provider.

```
// ThemeContext.jsx
import { createContext, useState } from 'react';

// Criação do contexto
export const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  function toggleTheme() {
    setTheme(prevTheme =>
      prevTheme === 'light' ? 'dark' : 'light'
    );
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

O que é um Provider?

O Provider é um **componente especial que "fornece" os dados do contexto** para todos os seus componentes filhos na árvore.

Através da propriedade **value**, o Provider disponibiliza estados, funções ou qualquer outro dado que você queira compartilhar.

```
// App.jsx
import { ThemeProvider } from './ThemeContext';
import HomePage from './HomePage';

function App() {
  return (
    <ThemeProvider>
      <HomePage />
    </ThemeProvider>
  );
}

export default App;
```

Consumindo um Contexto

useContext Hook

A forma mais moderna e simples de consumir um contexto é usando o hook **useContext**:

```
// Componente.jsx
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Componente() {
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <div>
      <p>O tema atual é: <strong>{theme}</strong></p>
      <button onClick={toggleTheme}>Alternar
Tema</button>
    </div>
  );
}

export default Componente;
```

Context Consumer (método alternativo)

Você também pode usar o padrão de Consumer, embora seja menos comum com os hooks:

```
// Componente.jsx
import { ThemeContext } from './ThemeContext';

function Componente() {
  return (
    <ThemeContext.Consumer>
      ({ { theme, toggleTheme } } => (
        <div>
          <p>O tema atual é: <strong>{theme}</strong></p>
          <button onClick={toggleTheme}>Alternar Tema</button>
        </div>
      ))
    </ThemeContext.Consumer>
  );
}

export default Componente;
```

Boas Práticas com Context API

Separar Contextos por Responsabilidade

Crie contextos diferentes para assuntos distintos: `UserContext`, `ThemeContext`, `CartContext`, etc.

Isso facilita a manutenção e evita renderizações desnecessárias quando apenas um dos valores muda.

Nomenclatura Clara e Organização

Use nomes descritivos para seus contextos e organize-os em uma estrutura de pastas consistente:

- `contexts/ThemeContext.jsx`
- `contexts/UserContext.jsx`

Escopo Limitado

Não envolva toda a aplicação em um contexto sem necessidade. Contextos têm **custo de performance**, pois podem causar renderizações em cascata.

Utilize-os apenas nos componentes que realmente precisam dos dados.

Exportação Separada

Sempre exporte o contexto e o provider separadamente para permitir maior flexibilidade de uso:

- `export const ThemeContext = ...`
- `export function ThemeProvider({ children }) ...`

Lembre-se: a Context API não substitui gerenciadores de estado como Redux para aplicações complexas, mas é uma solução excelente para compartilhamento de dados em escopo médio.